

PYNQcast: A Multiplayer Raycasting Game Engine on PYNQ FPGA with Cloud-Hosted Development Environment

A. Kendall | A. Shah | A. Tariq | L. Butler-Cassar | L. Canning | N. Himaz | S. Amos-Osebeyo
Imperial College London | March 2026

Abstract

PYNQcast is a multiplayer first-person raycasting graphics engine packaged with a cloud-hosted development and game hosting platform built on an IoT architecture. PYNQ-Z1 FPGA nodes render scenes entirely in synthesised hardware, synchronising with a cloud-authoritative EC2 server over UDP. The server resolves game state across all nodes and exposes a browser-based development environment; demonstrating hardware-accelerated rendering, bidirectional multi-node communication, and cloud-authoritative state management within an IoT framework. PYNQcast explores whether a resource-constrained FPGA SoC can deliver real-time multiplayer graphics with cloud-authoritative game state, framed as an IoT system where nodes capture input, process it locally in hardware, and exchange bidirectional state with a central server.

1 System Architecture & Overview

PYNQcast is built across two or more PYNQ-Z1 FPGA nodes and an AWS EC2 server (Figure 1). Each node's programmable logic (PL) runs a hardware Digital Differential Analyser (DDA) raycasting engine outputting directly to HDMI; the processing system (PS) handles button input, computes player position and camera angle, and synchronises state with the server over UDP. A shared BRAM bridges the PS/PL boundary. The EC2 server is authoritative for all game state; resolving collisions, broadcasting unified state back to nodes each tick, and exposing a browser-based development environment for map editing, session management, and match replays.

ric in the PL. A DDA raycasting engine runs on the PL, exploiting the parallelism of FPGA fabric for per-column ray computation, while a Python game script runs on the PS, handling player input, game state, and network communication with the game server.

The PS/PL boundary is bridged via a shared BRAM, exposed to the PS through an AXI interconnect. Each game tick, the PS writes the player coordinates, camera angle, and remote player position into BRAM; the PL consumes these and the map layout each frame to drive the raycasting engine. Player input is handled via an AXI GPIO peripheral mapped to the four onboard buttons.

2.2 FPGA Rendering Pipeline

2.2.1 DDA Algorithm Overview

As shown in Figure 2, stepping along a ray at fixed increments risks skipping over walls. DDA instead jumps from one grid boundary to the next, advancing to the nearest horizontal or vertical tile edge until a wall is hit. The algorithm relies heavily on trigonometric and reciprocal functions, this was the primary target for hardware acceleration in the PL implementation.

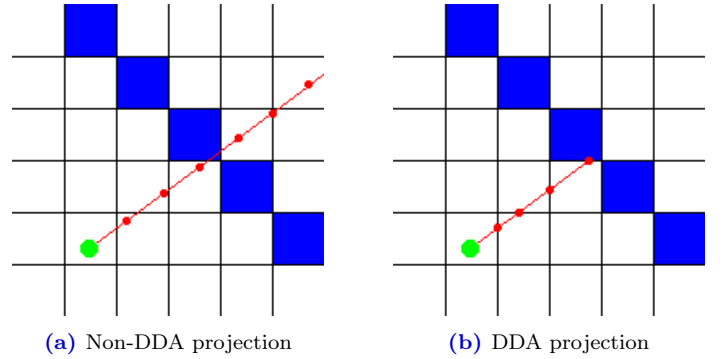


Figure 2: Ray projection comparison.

Frame Rate: To create the illusion of continuous movement, the renderer must sustain at least 60 FPS, the conventional threshold at which individual frames become imperceptible. The raycasting engine is designed to meet and exceed this, rendering frames directly from BRAM as fast as the pixel clock allows. The clean PS/PL boundary also simplified adding features such as game replay: position data is streamed from the server and written into BRAM, leaving the raycaster design entirely unchanged.

2.2.2 Raycasting Engine Inputs

The raycaster requires the values of 4 states to properly render a frame:

- **A 32×32 bit map:** stored in the first 32 addresses of the BRAM as 32 bit words, where each bit represents a 'walled' cell (1), or an open cell (0).

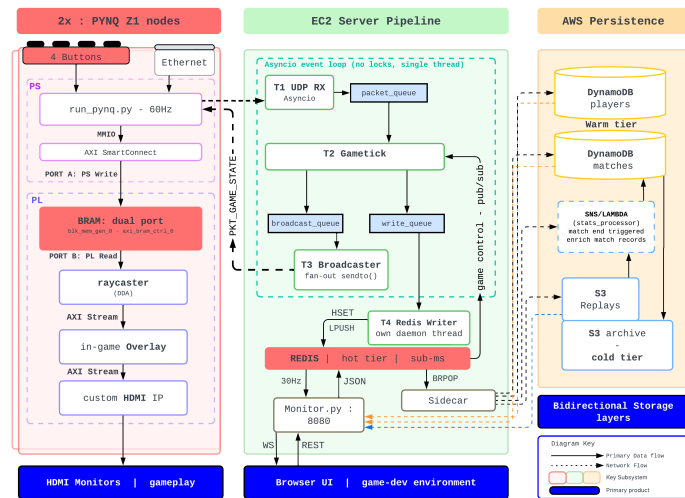


Figure 1: Top-level system architecture.

2 Client Node Design

2.1 SoC Architecture

The client-node is built around the PYNQ-Z1 SoC, coupling a dual-core ARM Cortex-A9 PS with Zynq-7000 FPGA fab-

- **The player coordinates:** these are represented in a 32 bit word, the most significant 16 bits correspond to the X coordinate and the least significant bits represent the Y coordinate - both in Q6.10.
- **The player viewing angle:** Encoded using 12 bit Binary Angle Measurement (BAM), where $0x000$ represents 0, and $0xFF$ is a full rotation (anticlockwise). This is beneficial as natural integer overflow perfectly emulates a full rotation, eliminating explicit wraparound logic in hardware.
- **Sprite coordinates:** This can represent another player or object, the sprite coordinates use the same format as the user coordinates.

2.2.3 Implementation in hardware

The principal implementation challenge was mapping the DDA arithmetic into synthesisable fixed-point logic. Each signal's value range was analysed independently to select the optimal representation. For example, unsigned Q6.10 for distances and coordinates (covering the 32×32 map's maximum diagonal of ≈ 45 units with 10 bits of fractional precision) and two's complement Q2.14 for unit direction vectors (maximising precision over $[-1, 1]$). Bit widths were constrained to 16 bits throughout to minimise DSP and adder usage, keeping all arithmetic combinatorial.

For computationally intensive operations such as $\sin(x)$, $\cos(x)$, and division, which are difficult to implement in hardware without sacrificing precision or throughput, custom lookup tables (LUTs) were employed.

LUT Design Tradeoffs: Three considerations drove the LUT approach. First, **latency:** LUTs provide single-cycle access, whereas a hardware divider via DSPs could require multiple cycles of iterative computation, introducing stalls in a per-column rendering loop. Second, **resource utilisation:** storing values in BRAM preserves DSP slices for any future compute demand. Third, **precision control:** LUT addressing allows independent tuning of input and output precision, so table sizes were minimised by analysing exactly how much fractional resolution each downstream computation required.

2.2.4 AXI Video stream output block & pixel generation

Whenever the distance of a ray is calculated it indexes the $\frac{240}{\text{distance}}$ LUT to compute the wall height in the corresponding column. A pulse is sent to the HDMI pixel gen block so that knows to store this height in its array of column heights.

At the start of each frame, the pixel generation block latches the column heights array. As pixels are output sequentially, the current screen coordinates determine which column is active, its wall height, and therefore whether the current pixel lies within the wall, above it (ceiling), or below it (floor), selecting the appropriate pixel value accordingly.

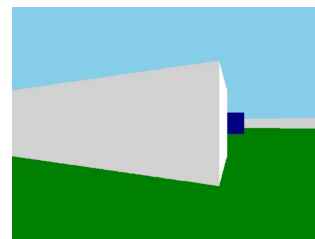
The AXI stream feeds directly into the custom HDMI interface block, driving the monitor without requiring a pixel buffer, avoiding the significant FF resource cost this would incur.

2.3 Extension 1: Sprite Rendering

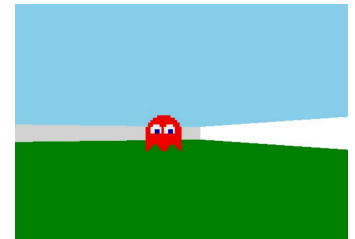
Sprites and wall textures are fully customisable by replacing the relevant .mem files. Sprite rendering proceeds in five stages:

1. Record the perpendicular wall distance for each vertical stripe in a 1D Z-buffer during the wall pass.
2. Project each sprite onto the camera plane by subtracting the player position and multiplying by the inverse camera matrix.
3. Determine on-screen sprite dimensions from its perpendicular distance.
4. Render column by column, discarding any column occluded by the Z-buffer.
5. Within each column, apply a transparent colour key to avoid rendering sprites as solid rectangles.

Sprite casting only needs one calculation per frame, hence it can run in parallel with wall raycasting, using multi-cycle dividers for camera transformations without impacting throughput. Sprites are stored as 16×16 12-bit RGB bitmaps with a transparency mask, enabling non-rectangular sprite shapes.



(a) Step 1 : Correctly rendering a square sprite in environment (with occlusion)

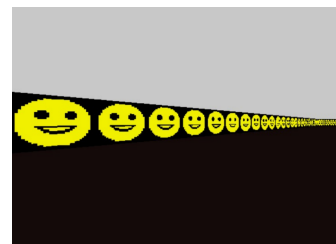


(b) Step 2 : Rendering sprite bitmaps correctly - in this case a Pac-Man-style ghost

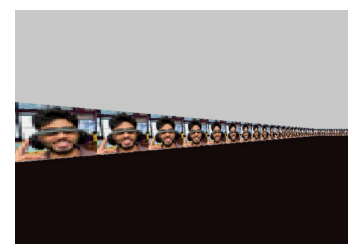
Figure 3: Sprite rendering - key visual components

2.4 Extension 2 : Texture Rendering

Texture rendering extends the core raycasting pipeline with wall-surface sampling. The X position within a wall is recorded at the end of DDA stepping; the Y position is resolved via a reciprocal LUT lookup to determine the fractional depth within the wall column. Together these index a 32×32 12-bit RGB bitmap, stored at reduced bit-width to save area, which is zero-extended to 24-bit for output.



(a) Simple smiley face bitmap.



(b) Picture of group member Adil to showcase complex image texture rendering.

Figure 4: Texture mapping: loading different textures.

2.5 HDMI Output Protocol

The default PYNQ video overlay routes every pixel through DDR3. The PS writes a complete frame into SDRAM, a VDMA engine reads it back over AXI, and the VTC clocks it through AXI4-Stream to Video Out and RGB-to-DVI. Buffering a full 480p frame costs $640 \times 480 \times 3 = 900$ kB per frame, and at 60 fps the VDMA alone consumes 55 MB/s of AXI bandwidth. For a real-time game engine this is unacceptable: frame rate couples to PS scheduling, and the display can scan out a frame while the PS is still writing the next one into the same buffer, causing tearing.

The VDMA is removed entirely. The raycaster AXI-Stream output connects directly into `v_axi4s_vid_out_0`, with the VTC and RGB-to-DVI IP retained unchanged. The VTC is a hardware counter driven by a fixed pixel clock, so frame rate is governed entirely by hardware and tearing has no shared framebuffer to occur on.

End-to-end pixel latency drops from **16.7 ms** to under **1 μ s**, the full 55 MB/s AXI bandwidth is recovered for networking, and the ARM core is reduced to writing a game-state packet to BRAM each tick rather than managing DMA transfers.

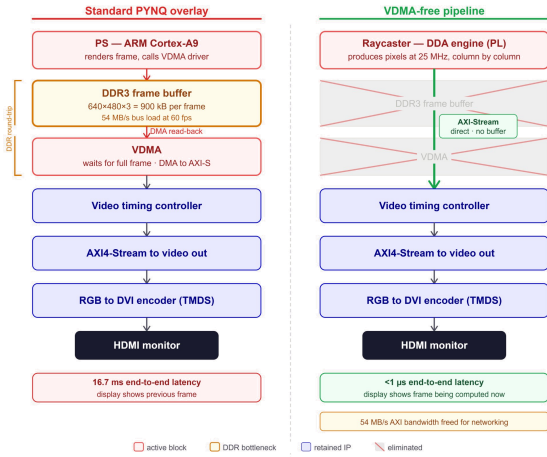


Figure 5: Standard PYNQ overlay vs. VDMA-free pipeline.

Design note: Any future PL rendering block can be substituted into this pipeline and drive the display without modifying the PS at all. The retained VTC, AXI4S-to-Video-Out, and RGB-to-DVI IPs occupy approximately 1 800 LUTs and zero BRAM tiles.

2.6 Testing

Tests were designed iteratively throughout raycaster development. The following three metrics were used to evaluate the performance of the raycaster design:

- **Latency:** The raycaster uses a double-buffered pipeline, DDA traversal completes in $\frac{1}{12}$ of the HDMI output period, making the display the throughput bottleneck. At 60 FPS the renderer operates at under 10% capacity, theoretically supporting >600 FPS unconstrained. BRAM state updates are picked up within one render cycle, giving sub-frame input-to-display latency.
- **Resource Utilisation:** The complete PL design, including the AXI interconnect, BRAM controllers, and all custom IP, totals approximately 20% utilisation, leaving headroom for future feature additions.
- **Image Quality:** Correct perspective rendering with no fisheye distortion. 16×16 sprites render with correct wall occlusion; 32×32 wall textures are accurately mapped.

These metrics were chosen because they capture the three constraints that govern a real-time hardware renderer; whether the system meets human-perceptual timing thresholds, whether the design fits within the finite FPGA fabric leaving headroom for extension, and whether the rendering output is geometrically correct.

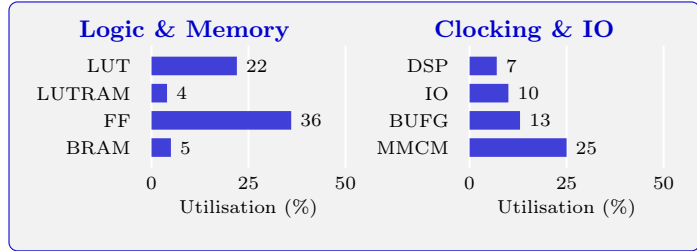
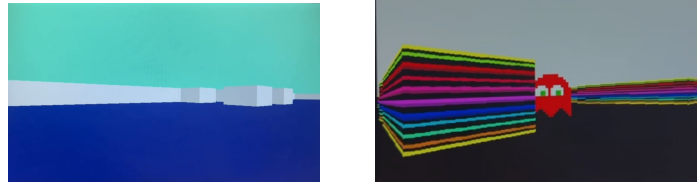


Figure 6: Post-Implementation Resource Utilisation.



(a) Shaded walls. (b) Sprite occlusion.

Figure 7: Texture and sprite rendering progression.

Hardware Verification Without Synthesis: Two complementary techniques avoided the time cost of repeated synthesis and bitstream generation. First, a Python script rendered frames directly from the SystemVerilog simulation output, giving immediate visual feedback on image correctness. Second, key internal signals were surfaced to the top level, allowing the Vivado waveform viewer to verify signal propagation through the pipeline at any stage.

2.7 Extension: UI Overlay

To give developers building on the platform full freedom to design their own interface, the engine exposes a general-purpose compositing layer requiring no bitstream changes between applications, only different register values written by the PS at startup.

Two IPs are chained in the pixel stream at 74.25 MHz. The **textbox IP** evaluates eight axis-aligned rectangle slots in fully parallel combinatorial logic each pixel clock cycle. Each slot stores textbox info in AXI-Lite registers. The logic performs comparisons per slot to detect whether the beam falls within the bounding rectangle with a priority mux selecting the first hit. The result is latched in a single register stage giving one cycle of pipeline latency. **Loading bars** follow the same per-pixel evaluation principle and are backed by a fill amount written by the PS.

The **text overlay** reads from a glyph BRAM covering 96 printable ASCII characters. Figure 8 shows how the beam position resolves to a BRAM address, a fetched byte, and a one-bit gate in a single clock cycle.

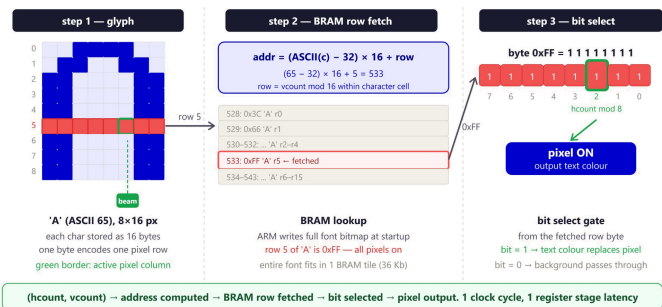


Figure 8: Glyph ROM addressing for 'A' at scan row 5: beam position drives a BRAM lookup returning `0xFF`, from which a single bit gates the pixel output in one clock cycle.

Design note: The glyph BRAM doubles as a sprite tile store: the PS uploads a custom bitmap at runtime and the same one-cycle path composites it against a register-backed colour table, extending text rendering to arbitrary tile graphics at no additional hardware cost. Together the IPs consume ~ 180 LUTs and 1 BRAM tile, versus ~ 55 MB/s of AXI bandwidth for a software equivalent at 480p.

3 Server Architecture

The EC2 game server is a single-threaded `asyncio` reactor whose processing is decomposed into four stages (T1–T4) (Figure 9) connected by lock-free queues, inspired by the staged event-driven topology of SEDA [3]. Unlike classical SEDA, there are no thread pools: the hot path is I/O-bound with negligible CPU work per tick, so threading would add lock contention and context-switch overhead for no throughput gain. T1–T3 therefore run as cooperative coroutines on one OS thread; only T4 (blocking Redis writes) requires a second thread. This gives the structural benefits of staged decomposition (per-stage observability, backpressure isolation, independent testability) without shared-state locking.

T1 (`DatagramProtocol` callback) enqueues each UDP datagram on `packet_queue`.

T2 (game tick) drains the queue each tick, runs authoritative game logic, builds the outbound `PKT_GAME_STATE`, and pushes it to `broadcast_queue` (T3) and `write_queue` (T4). A monotonic self-correcting scheduler compensates for event-loop drift.

T3 awaits `broadcast_queue` and calls `transport.sendto()` for each node.

T4 blocks on `write_queue.get()`, batches all HSET operations into a single Redis pipeline per tick.

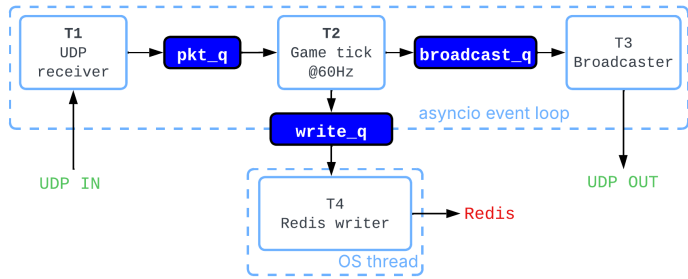


Figure 9: The Server Reactor pipeline with SEDA (Staged Event Driven Architecture) inspired topology.

Authority model: Nodes are authoritative for position; the server is authoritative for all match events. Sequence validation drops stale/replayed packets via 16-bit circular comparison. Wall-collision clamping snaps illegal positions using axis-aligned slide with binary-search fallback. Ghost AI scores 10 candidate headings per tick by proximity progress and steer-change penalty: at $n=4$ entities the $O(n^2)$ proximity loop is negligible.

3.1 Wire Protocol

All packets are custom fixed-size `struct`-packed datagrams with no variable-length fields on the hot path. For example, the 24-byte `NodePacket` (<HHIffB3x>, Table 1) is transmitted once per tick, little-endian native on both ARM Cortex-A9 and x86-64. Aside from other packets, the server broadcasts a 68 B `PKT_GAME_STATE` each tick, and a 1032 B `PKT_MAP` once at match start to program the FPGA BRAM.

Table 1: `NodePacket`: 24-byte fixed binary frame.

Off	Size	Type	Field	Purpose
0	2 B	u16	<code>node_id</code>	Source identity
2	2 B	u16	<code>seq</code>	Monotonic counter
4	4 B	u32	<code>tick</code>	Server tick stamp
8	4 B	f32	<code>x</code>	World X
12	4 B	f32	<code>y</code>	World Y
16	4 B	f32	<code>angle</code>	Heading (rad)
20	1 B	u8	<code>flags</code>	Input bitmask
21	3 B	—	<code>pad</code>	Alignment (3x)

3.2 Storage and Persistence

Game state flows through a three-tier cache hierarchy (Table 2). Redis holds the live game state that T2 and the monitor poll every tick. At match end, the sidecar promotes event data to DynamoDB for queryable match history. Completed matches are then drained to S3 as compressed replays. The critical constraint is that DynamoDB write latency (5–20 ms) exceeds the 16.67 ms tick budget, so all AWS I/O is isolated in the sidecar process.

Table 2: Storage hierarchy: each tier feeds the next.

Tier	Service	Latency	Role
Hot	Redis	<1 ms	Live state, event queues, Pub/Sub control
Warm	DynamoDB	5–20 ms	Match metadata, player profiles (atomic ADD)
Cold	S3	50–200 ms	Compressed NDJSON replays, DDB archives

The **sidecar** (`sidecar.py`) bridges the tiers: it consumes Redis event queues via BRPOP, writes match records to DynamoDB, streams replay data to S3 via multipart upload (5 MB gzip parts), and enforces warm-tier retention by archiving old DynamoDB rows to S3. Redis is the boundary: T4 pushes with LPUSH; the sidecar pops asynchronously. At match end, DynamoDB and SNS operations run in a background thread so the BRPOP loop unblocks immediately for the next match.

3.3 Monitor: Game Development Environment

The browser-based monitor distinguishes PYNQcast as a *game development environment*. Commands are published via HTTP POST to Redis Pub/Sub (`game:control`); T2 subscribes and drains the control queue each tick.

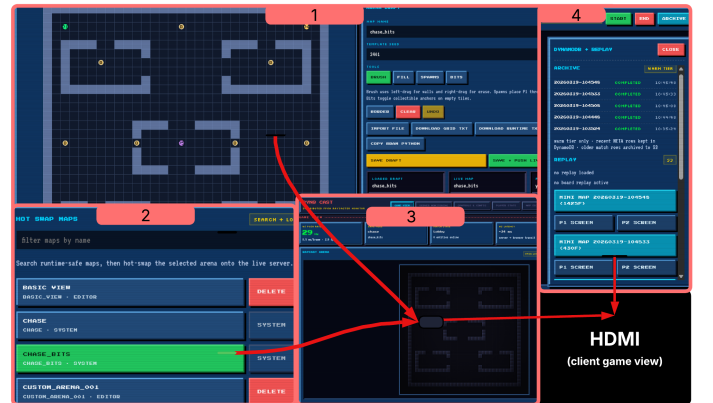


Figure 10: Monitor dashboard panes built in HTML, CSS, JS and React. Served by SSH tunnel from EC2. (1) Map editor (2) Map library (3) Live arena with all players (4) Match Replay activation from S3. Some standout panels are shown in Figure 10 : (1) the map editor with brush, fill, and spawn tools for designing 32×32 arenas; (2) the map library for live hot-swapping of

stored arenas; (3) the live game top-down view with the new geometry rendered; and (4) the match archive and replay panel, which can stream stored matches directly to a specific player’s screen. Traits for AI entities can also be configured live.

Every match is automatically recorded via the sidecar’s S3 replay stream; “P1/P2 Screen” button fetches the file, switches target boards into replay mode via PKT_NODE_MODE, and streams each frame as a standard PKT_GAME_STATE at the original tick rate. The FPGA re-executes original game geometry from stored state.

3.4 Evaluation

3.4.1 Test Suite

The server-side logic is validated by ~150 `pytest` tests. Tests cover three concerns: **game rules**, **protocol integrity** and **hardware contract**. An in-process tick benchmark asserts average cycle time <25 ms and P95 <35 ms, confirming 60 Hz budget headroom. This is an example of one test subsystem that fits into our larger system testing flow (Figure 11).

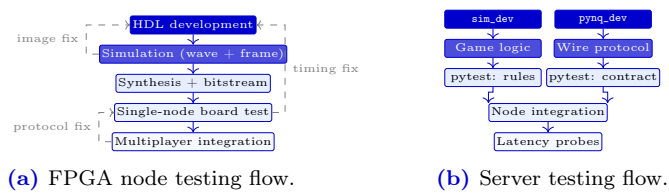


Figure 11: System testing flow: parallel FPGA and server tracks converging at integration.

3.5 Latency Benchmarks

To validate that the authoritative server model adds negligible lag, measurements were taken on physical PYNQ Z1’s against EC2 eu-west-2 over 5G hotspot. **RTT mode** sends a bare UDP echo probe isolating the network path. **Button-to-visible (B2V) mode** times the vital full player-perceived path. Subtracting RTT P50 from B2V P50 isolates server overhead at 1.70 ms (5% of total).

Figure 12 maps tests onto the architecture. The B2V waterfall decomposes into network (~17 ms each way, 95%) and server processing (1.70 ms, 5%). The persistence panel shows decoupled I/O that never blocks the player path.

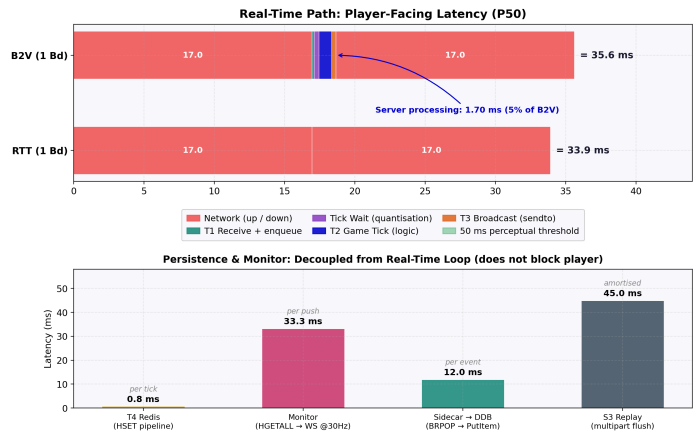


Figure 12: Latency budget. Top: player-facing path with server overhead at 5%. Bottom: decoupled persistence and monitor I/O.

Figure 13 shows B2V under increasing load and shows that the server acts uniformly even with all environment variables

in action.

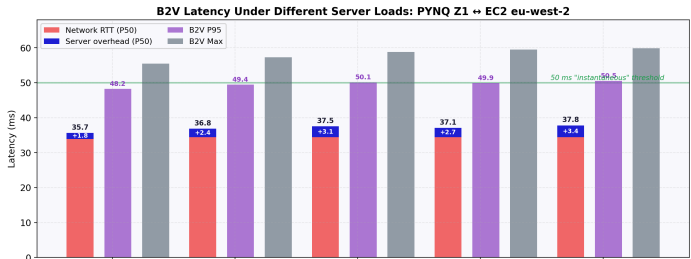


Figure 13: B2V under load. Server overhead <4 ms in all cases; P95 stays near 50 ms.

Figure 14 shows the three inter-stage queue depths over 10 s with 2 boards and 2 AI ghosts. All queues hover near zero (mean <0.5, max 4). No sustained growth confirms every stage drains faster than work arrives. This per-stage visibility is a direct benefit of the staged reactor topology, the queue-connected pipeline makes bottlenecks immediately attributable to a specific stage.

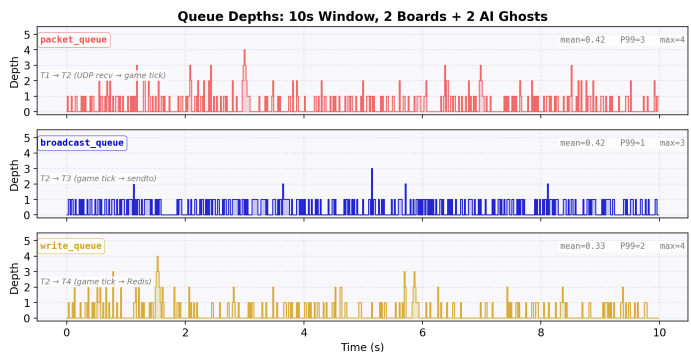


Figure 14: Queue depths: 10 s, 2 boards + AI. Mean <0.5, no backpressure.

4 Conclusion

PYNQcast demonstrates a complete IoT gaming platform meeting all minimum functional requirements: local FPGA processing, cloud server deployment, bidirectional node-server communication, and verified multi-node operation. Key contributions include a fully pipelined hardware DDA raycaster operating at under 10% capacity at 60 FPS and a pipelined-reactor server sustaining <4 ms overhead under full load. Future work includes inline overlay integration and extension to additional node types.

References

[1] Abrash, M., *Graphics Programming Black Book*, 1997.

[2] Xilinx, *PYNQ: Python Productivity for Zynq*, 2021.

[3] Welsh, M. et al., “SEDA: An Architecture for Well-Conditioned, Scalable Internet Services,” *SOSP*, 2001.

[4] Van der Linden, L., “Lode’s Computer Graphics Tutorial: Raycasting,” *lodev.org*, 2004. [Online]. Available: <https://lodev.org/cgtutor/raycasting.html>

Appendix

- Repository: <https://github.com/PYNQcast>
- Demo Video: https://www.youtube.com/watch?v=t1CdY_SkK-g